

Q1: Write a Python Program to implement Breadth First Search.

```
from collections import deque

def bfs(graph, start_node):
    """
    Implements the Breadth-First Search algorithm.

    Args:
        graph (dict): A dictionary representing the graph where keys are nodes
                      and values are lists of their adjacent nodes.
        start_node: The starting node for the BFS traversal.

    Returns:
        list: A list containing the nodes in the order they were visited by BFS.
    """
    visited = set() # To keep track of visited nodes
    queue = deque() # Queue for BFS traversal

    # Add the starting node to visited and the queue
    visited.add(start_node)
    queue.append(start_node)

    traversal_order = [] # To store the order of visited nodes

    while queue:
        current_node = queue.popleft() # Dequeue a node
        traversal_order.append(current_node)

        # Explore neighbors
        for neighbor in graph.get(current_node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return traversal_order

# Example Usage:
if __name__ == "__main__":
    # Representing a graph using an adjacency list
    graph = {
        'A': ['B', 'C'],
        'B': ['D', 'E'],
        'C': ['F'],
        'D': [],
        'E': ['F'],
        'F': []
    }

    print("BFS traversal starting from 'A':")
    result = bfs(graph, 'A')
    print(result) # Expected output: ['A', 'B', 'C', 'D', 'E', 'F']

    print("\nBFS traversal starting from 'B':")
    result_b = bfs(graph, 'B')
    print(result_b) # Expected output: ['B', 'D', 'E', 'F']
```

```
BFS traversal starting from 'A':  
['A', 'B', 'C', 'D', 'E', 'F']  
  
BFS traversal starting from 'B':  
['B', 'D', 'E', 'F']
```

Q2: Write a Python Program to implement Min-Max Algorithm.

```
import math  
  
def minimax(current_depth, node_index, is_maximizing_player, scores, target_depth):  
    """  
    Implements the Min-Max algorithm.  
  
    Args:  
        current_depth (int): The current depth in the game tree.  
        node_index (int): The index of the current node in the scores array.  
        is_maximizing_player (bool): True if it's the maximizing player's turn, False  
otherwise.  
        scores (list): A list of scores representing the terminal states (leaf nodes).  
        target_depth (int): The maximum depth of the game tree.  
  
    Returns:  
        int: The optimal score for the current player from this node.  
    """  
  
    # Base case: target_depth reached (leaf node)  
    if current_depth == target_depth:  
        return scores[node_index]
```

```

    if is_maximizing_player:
        # Maximizing player chooses the maximum of the child nodes
        left_child_score = minimax(current_depth + 1, node_index * 2, False, scores,
target_depth)
        right_child_score = minimax(current_depth + 1, node_index * 2 + 1, False,
scores, target_depth)
        return max(left_child_score, right_child_score)
    else:
        # Minimizing player chooses the minimum of the child nodes
        left_child_score = minimax(current_depth + 1, node_index * 2, True, scores,
target_depth)
        right_child_score = minimax(current_depth + 1, node_index * 2 + 1, True,
scores, target_depth)
        return min(left_child_score, right_child_score)

# Driver code
if __name__ == "__main__":
    # Example scores for a game tree with depth 3 (8 leaf nodes)
    # The scores are ordered as if read from left to right in the tree's leaf level.
    scores = [3, 5, 2, 9, 12, 5, 23, 23]

    # Calculate the depth of the tree (log base 2 of the number of leaf nodes)
    # For a complete binary tree, number of leaf nodes = 2^depth
    # So, depth = log2(number of leaf nodes)
    tree_depth = int(math.log2(len(scores)))

    print("The optimal value is:", minimax(0, 0, True, scores, tree_depth))

```

The optimal value is: 12

Q3: Write a Python Program to implement the Backtracking approach to solve N Queen's problem

```

def solve_n_queens(n):
    """
    Finds all distinct solutions to the N-Queens puzzle.

    Args:
        n (int): The size of the chessboard.
    """
    results = []
    # Set to store the occupied columns
    cols = set()
    # Sets to store the occupied diagonals
    # A positive diagonal (top-left to bottom-right) has a constant (row + col)
    pos_diagonals = set()
    # A negative diagonal (top-right to bottom-left) has a constant (row - col)
    neg_diagonals = set()

    # Initialize the chessboard
    board = [['.' for _ in range(n)] for _ in range(n)]

    def backtrack(col):
        # Base case: All queens have been placed
        if col == n:
            # A solution is found, so add it to the results list
            solution = ["".join(row) for row in board]
            results.append(solution)
            return

        # Iterate through each row of the current column
        for row in range(n):
            # Check if the position (row, col) is safe for a queen
            if row in cols or (row + col) in pos_diagonals or (row - col) in neg_diagonals:
                continue

            # Place the queen
            board[row][col] = 'Q'
            cols.add(row)
            pos_diagonals.add(row + col)
            neg_diagonals.add(row - col)

            # Recur for the next column
            backtrack(col + 1)

            # Backtrack by undoing the placement
            board[row][col] = '.'
            cols.remove(row)
            pos_diagonals.remove(row + col)
            neg_diagonals.remove(row - col)

    backtrack(0)
    return results

def print_solutions(solutions):
    """
    Prints all the found solutions in a readable format.

    Args:
        solutions (list): A list of valid board configurations.
    """
    if not solutions:
        print("No solution exists.")
        return

```

```

print(f"Found {len(solutions)} solutions.")
for i, solution in enumerate(solutions):
    print(f"Solution {i+1}:")
    for row in solution:
        print(row)
    print()

# --- Example Usage ---
if __name__ == "__main__":
    n = 4
    solutions = solve_n_queens(n)
    print_solutions(solutions)

    n = 8
    solutions = solve_n_queens(n)
    print_solutions(solutions)

```

```
Found 2 solutions.
```

```
Solution 1:
```

```
..Q.
```

```
Q...
```

```
...Q
```

```
.Q..
```

```
Solution 2:
```

```
.Q..
```

```
...Q
```

```
Q...
```

```
..Q.
```

Q4: Write a Python Program to implement A* Algorithm.

```

import heapq

class Node:
    """
    Represents a node in the grid for A* algorithm.
    """
    def __init__(self, x, y, g=0, h=0, parent=None):
        self.x = x
        self.y = y
        self.g = g # Cost from start to this node
        self.h = h # Heuristic cost from this node to goal
        self.f = g + h # Total cost (g + h)
        self.parent = parent # Parent node in the path

    def __lt__(self, other):
        """
        Comparison for priority queue (heapq).
        Nodes are ordered by their f-cost.
        """
        return self.f < other.f

def heuristic(node_a, node_b):
    """
    Manhattan distance heuristic.
    Calculates the estimated cost from node_a to node_b.
    """
    return abs(node_a.x - node_b.x) + abs(node_a.y - node_b.y)

def a_star(grid, start, end):
    """
    Implements the A* search algorithm.

```

```

Args:
    grid (list of lists): A 2D list representing the grid.
                          0 for traversable, 1 for obstacle.
    start (tuple): (x, y) coordinates of the starting point.
    end (tuple): (x, y) coordinates of the ending point.

Returns:
    list of tuples: The shortest path from start to end, or None if no path exists.
"""
rows = len(grid)
cols = len(grid[0])

start_node = Node(start[0], start[1])
end_node = Node(end[0], end[1])

open_list = [] # Priority queue for nodes to be evaluated
heapq.heappush(open_list, start_node)

closed_list = set() # Set of nodes already evaluated

# Dictionary to store the best path to each node
# Key: (x, y) tuple, Value: Node object
came_from = {}

g_scores = { (start_node.x, start_node.y): 0 } # Cost from start to current node
f_scores = { (start_node.x, start_node.y): start_node.h } # Total estimated cost

while open_list:
    current_node = heapq.heappop(open_list)

    if (current_node.x, current_node.y) == (end_node.x, end_node.y):
        # Path found, reconstruct it
        path = []
        while current_node:
            path.append((current_node.x, current_node.y))
            current_node = current_node.parent
        return path[::-1] # Reverse to get path from start to end

    closed_list.add((current_node.x, current_node.y))

    # Explore neighbors
    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]: # 4-directional movement
        neighbor_x, neighbor_y = current_node.x + dx, current_node.y + dy

        # Check if neighbor is within grid bounds and not an obstacle
        if 0 <= neighbor_x < rows and 0 <= neighbor_y < cols and grid[neighbor_x][neighbor_y] == 0:
            neighbor_node = Node(neighbor_x, neighbor_y)

            if (neighbor_x, neighbor_y) in closed_list:
                continue

            tentative_g_score = current_node.g + 1 # Assuming cost of 1 for each step

            if (neighbor_x, neighbor_y) not in g_scores or tentative_g_score < g_scores[(neighbor_x, neighbor_y)]:
                neighbor_node.parent = current_node
                neighbor_node.g = tentative_g_score
                neighbor_node.h = heuristic(neighbor_node, end_node)
                neighbor_node.f = neighbor_node.g + neighbor_node.h
                g_scores[(neighbor_x, neighbor_y)] = tentative_g_score
                f_scores[(neighbor_x, neighbor_y)] = neighbor_node.f

                if (neighbor_x, neighbor_y) not in [(node.x, node.y) for node in open_list]:
                    heapq.heappush(open_list, neighbor_node)

```

```

# 0 represents traversable, 1 represents obstacle
grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

start_point = (0, 0)
end_point = (4, 4)

path = a_star(grid, start_point, end_point)

if path:
    print("Path found:")
    for x, y in path:
        print(f"({x}, {y})", end=" -> ")
    print("End")
else:
    print("No path found.")

# Visualize the path on the grid (optional)
if path:
    display_grid = [row[:] for row in grid] # Create a copy
    for x, y in path:
        if (x, y) != start_point and (x, y) != end_point:
            display_grid[x][y] = '*' # Mark path

    display_grid[start_point[0]][start_point[1]] = 'S'
    display_grid[end_point[0]][end_point[1]] = 'E'

    print("\nGrid with path:")
    for row in display_grid:
        print(" ".join(map(str, row)))

```

```

Path found:
(0, 0) -> (0, 1) -> (0, 2) -> (0, 3) -> (0, 4) -> (1, 4) -> (2, 4) -> (3, 4) -> (4, 4) -> End

Grid with path:
S * * * *
0 1 0 1 *
0 0 0 0 *
0 0 0 0 *
0 1 1 1 *
0 0 0 0 E

```

Q5: Write a Python Program to implement Naïve Bayes Algorithm for data classification, choose dataset of your own choice.

```

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load the dataset (Iris dataset)
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)

# 2. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 3. Initialize and train the Naïve Bayes model (Gaussian Naïve Bayes for continuous features)
model = GaussianNB()
model.fit(X_train, y_train)

# 4. Make predictions on the test set
y_pred = model.predict(X_test)

# 5. Evaluate the model
print("--- Naïve Bayes Classifier Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=iris.target_names))

```

```

--- Naïve Bayes Classifier Performance ---
Accuracy: 0.9778

```

```

Confusion Matrix:

```

```

[[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]

```

```

Classification Report:

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	0.92	0.96	13
virginica	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

Q6: Write a Python Program to implement Polynomial Regression on a dataset of your own choice.

```

#Three lines to make our compiler able to draw:
import sys
import matplotlib
matplotlib.use('Agg')

import numpy
import matplotlib.pyplot as plt

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

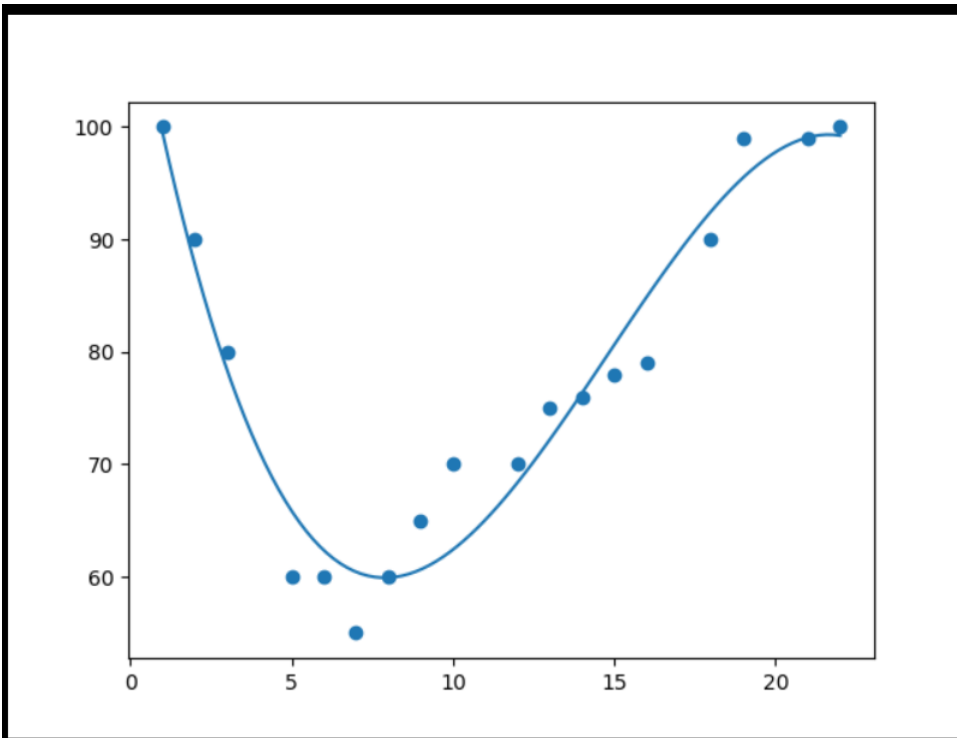
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

myline = numpy.linspace(1, 22, 100)

plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()

#Two lines to make our compiler able to draw:
plt.savefig(sys.stdout.buffer)
sys.stdout.flush()

```



Q7: Take a Data set as per your choice, implement and execute on different inputs of K-Means clustering algorithm.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from numpy.random import uniform
from sklearn.datasets import make_blobs
import seaborn as sns
import random

def euclidean(point, data):
    """
    Euclidean distance between point & data.
    Point has dimensions (m,), data has dimensions (n,m), and output will be of size (n,).
    """
    return np.sqrt(np.sum((point - data)**2, axis=1))

class KMeans:
    def __init__(self, n_clusters=8, max_iter=300):
        self.n_clusters = n_clusters
        self.max_iter = max_iter
    def fit(self, X_train):
        # Initialize the centroids, using the "k-means++" method, where a random datapoint is selected as the first,
        # then the rest are initialized w/ probabilities proportional to their distances to the first
        # Pick a random point from train data for first centroid
        self.centroids = [random.choice(X_train)]
        for _ in range(self.n_clusters-1):
            # Calculate distances from points to the centroids
            dists = np.sum([euclidean(centroid, X_train) for centroid in self.centroids], axis=0)
            # Normalize the distances
            dists /= np.sum(dists)

            | # Choose remaining points based on their distances
            new_centroid_idx, = np.random.choice(range(len(X_train)), size=1, p=dists)
            self.centroids += [X_train[new_centroid_idx]]
        # This initial method of randomly selecting centroid starts is less effective
        # min_, max_ = np.min(X_train, axis=0), np.max(X_train, axis=0)
        # self.centroids = [uniform(min_, max_) for _ in range(self.n_clusters)]
        # Iterate, adjusting centroids until converged or until passed max_iter
        iteration = 0
        prev_centroids = None
        while np.not_equal(self.centroids, prev_centroids).any() and iteration < self.max_iter:
            # Sort each datapoint, assigning to nearest centroid
            sorted_points = [[] for _ in range(self.n_clusters)]
            for x in X_train:
                dists = euclidean(x, self.centroids)
                centroid_idx = np.argmin(dists)
                sorted_points[centroid_idx].append(x)
            # Push current centroids to previous, reassign centroids as mean of the points belonging to them
            prev_centroids = self.centroids
            self.centroids = [np.mean(cluster, axis=0) for cluster in sorted_points]
            for i, centroid in enumerate(self.centroids):
                if np.isnan(centroid).any(): # Catch any np.nans, resulting from a centroid having no points
                    self.centroids[i] = prev_centroids[i]
            iteration += 1
        def evaluate(self, X):
            centroids = []
            centroid_idxs = []
            for x in X:
                dists = euclidean(x, self.centroids)
                centroid_idx = np.argmin(dists)
                centroids.append(self.centroids[centroid_idx])
                centroid_idxs.append(centroid_idx)
            return centroids, centroid_idxs

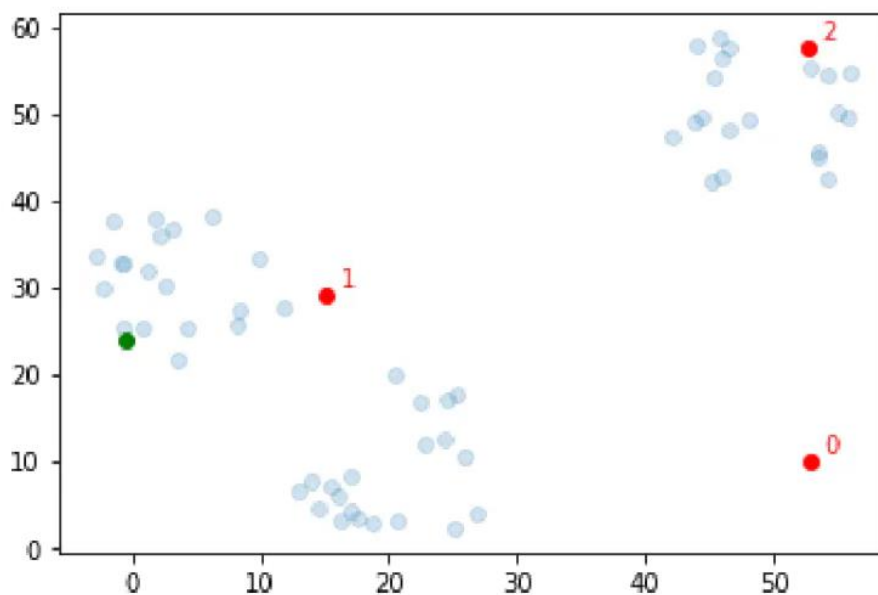
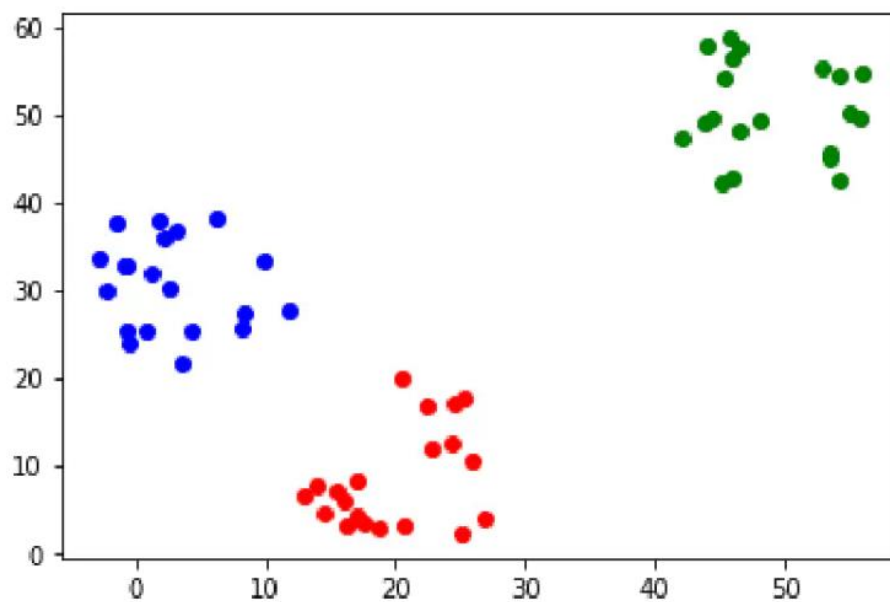
# Create a dataset of 2D distributions

```

```

# Create a dataset of 2D distributions
centers = 5
X_train, true_labels = make_blobs(n_samples=100, centers=centers, random_state=42)
X_train = StandardScaler().fit_transform(X_train)
# Fit centroids to dataset
kmeans = KMeans(n_clusters=centers)
kmeans.fit(X_train)
# View results
class_centers, classification = kmeans.evaluate(X_train)
sns.scatterplot(x=[X[0] for X in X_train],
                y=[X[1] for X in X_train],
                hue=true_labels,
                style=classification,
                palette="deep",
                legend=None
                )
plt.plot([x for x, _ in kmeans.centroids],
         [y for _, y in kmeans.centroids],
         'k+',
         markersize=10,
         )
plt.show()

```



Q8: Write a Python Program to implement FP tree growth Algorithm on a dataset of your own choice.

```
class FPTreeNode:
    def __init__(self, name, count, parent):
        self.name = name
        self.count = count
        self.parent = parent
        self.children = {}
        self.node_link = None # Link to the next node with the same item name

class FPGrowth:
    def __init__(self, min_support):
        self.min_support = min_support
        self.header_table = {}
        self.fp_tree = None

    def create_fp_tree(self, transactions):
        # First pass: Count item frequencies and filter infrequent items
        item_counts = {}
        for transaction in transactions:
            for item in transaction:
                item_counts[item] = item_counts.get(item, 0) + 1

        frequent_items = {item: count for item, count in item_counts.items() if count >= self.min_support}
        if not frequent_items:
            return None, None

        # Sort frequent items by frequency in descending order
        sorted_frequent_items = sorted(frequent_items.items(), key=lambda x: x[1], reverse=True)
        self.header_table = {item: [count, None] for item, count in sorted_frequent_items}

        # Initialize FP-tree
        self.fp_tree = FPTreeNode('null', 1, None)

    # Second pass: Build the FP-tree
    for transaction in transactions:
        # Filter and sort items in the transaction based on frequent items' order
        filtered_transaction = [item for item in transaction if item in frequent_items]
        filtered_transaction.sort(key=lambda item: frequent_items[item], reverse=True)

        if filtered_transaction:
            self._insert_transaction(filtered_transaction, self.fp_tree)

    return self.fp_tree, self.header_table

    def _insert_transaction(self, transaction, node):
        first_item = transaction[0]
        if first_item in node.children:
            node.children[first_item].count += 1
        else:
            new_node = FPTreeNode(first_item, 1, node)
            node.children[first_item] = new_node
            # Update header table link
            if self.header_table[first_item][1] is None:
                self.header_table[first_item][1] = new_node
            else:
                self._update_node_link(self.header_table[first_item][1], new_node)

        if len(transaction) > 1:
            self._insert_transaction(transaction[1:], node.children[first_item])

    def _update_node_link(self, head_node, new_node):
        while head_node.node_link is not None:
            head_node = head_node.node_link
        head_node.node_link = new_node
```

```

def find_frequent_itemsets(self, header_table, fp_tree):
    frequent_itemsets = []
    # Iterate through header table from least frequent to most frequent
    sorted_items = sorted(header_table.keys(), key=lambda item: header_table[item][0])

    for item in sorted_items:
        conditional_pattern_base = self._find_prefix_paths(item, header_table[item][1])
        if conditional_pattern_base:
            # Construct conditional FP-tree
            conditional_transactions = []
            for path, count in conditional_pattern_base:
                conditional_transactions.extend([path] * count)

            conditional_fp_growth = FPGrowth(self.min_support)
            conditional_tree, conditional_header = conditional_fp_growth.create_fp_tree(conditional_transactions)

            if conditional_tree and conditional_header:
                # Mine frequent itemsets from conditional FP-tree
                suffix = [item]
                self._mine_tree(conditional_tree, conditional_header, suffix, frequent_itemsets)

    # Add the single item if it's frequent
    if header_table[item][0] >= self.min_support:
        frequent_itemsets.append(frozenset([item]))

    return frequent_itemsets

def _find_prefix_paths(self, item_name, node):
    prefix_paths = []
    while node is not None:
        current_path = []

        parent_node = node.parent
        while parent_node.name != 'null':
            current_path.insert(0, parent_node.name)
            parent_node = parent_node.parent
        if current_path:
            prefix_paths.append((current_path, node.count))
        node = node.node_link
    return prefix_paths

def _mine_tree(self, tree, header, suffix, frequent_itemsets):
    # Iterate through header table from least frequent to most frequent
    sorted_items = sorted(header.keys(), key=lambda item: header[item][0])

    for item in sorted_items:
        new_frequent_set = frozenset(suffix + [item])
        frequent_itemsets.append(new_frequent_set)

        conditional_pattern_base = self._find_prefix_paths(item, header[item][1])
        if conditional_pattern_base:
            conditional_transactions = []
            for path, count in conditional_pattern_base:
                conditional_transactions.extend([path] * count)

            conditional_fp_growth = FPGrowth(self.min_support)
            conditional_tree, conditional_header = conditional_fp_growth.create_fp_tree(conditional_transactions)

            if conditional_tree and conditional_header:
                self._mine_tree(conditional_tree, conditional_header, suffix + [item], frequent_itemsets)

```

```

# Sample Dataset
transactions = [
    ['Milk', 'Onion', 'Nutmeg', 'Kidney Beans', 'Eggs', 'Yogurt'],
    ['Dill', 'Onion', 'Nutmeg', 'Kidney Beans', 'Eggs', 'Yogurt'],
    ['Milk', 'Apple', 'Kidney Beans', 'Eggs'],
    ['Milk', 'Unicorn', 'Corn', 'Kidney Beans', 'Yogurt'],
    ['Corn', 'Onion', 'Onion', 'Kidney Beans', 'Ice cream', 'Eggs']
]

min_support_threshold = 3 # Example minimum support

# Create and mine the FP-tree
fp_growth_instance = FPGrowth(min_support_threshold)
fp_tree, header_table = fp_growth_instance.create_fp_tree(transactions)

if fp_tree and header_table:
    frequent_itemsets = fp_growth_instance.find_frequent_itemsets(header_table, fp_tree)
    print("Frequent Itemsets:")
    for itemset in frequent_itemsets:
        print(itemset)
else:
    print("No frequent items found for the given minimum support.")

```

```

Frequent Itemsets:
frozenset({'Kidney Beans', 'Milk'})
frozenset({'Milk'})
frozenset({'Kidney Beans', 'Yogurt'})
frozenset({'Yogurt'})
frozenset({'Onion', 'Kidney Beans'})
frozenset({'Onion'})
frozenset({'Kidney Beans', 'Eggs'})
frozenset({'Onion', 'Eggs'})
frozenset({'Onion', 'Eggs', 'Kidney Beans'})
frozenset({'Eggs'})
frozenset({'Kidney Beans'})

```